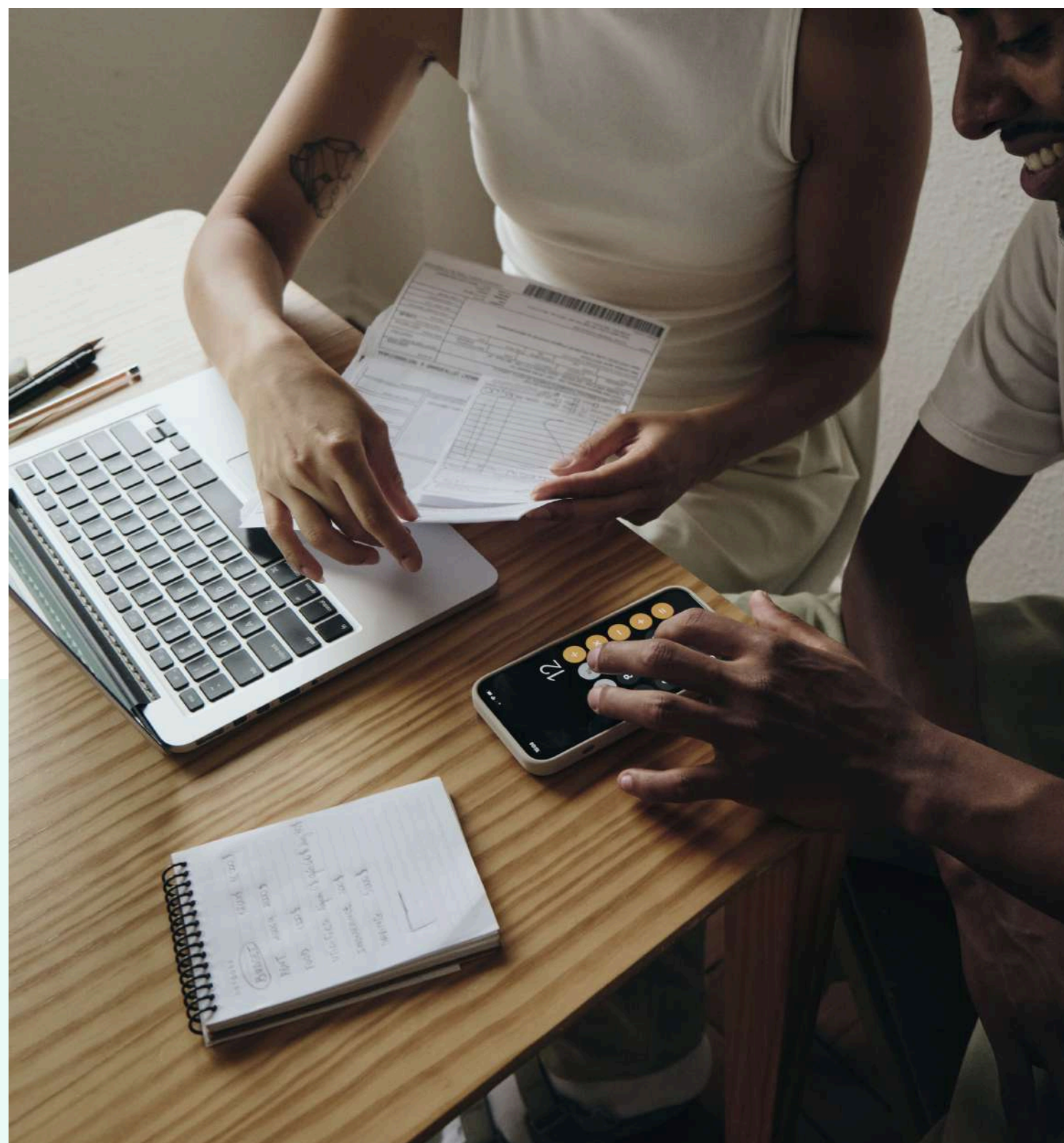


MISE EN PLACE D'UN MODÈLE DE SCORING CRÉDIT AUTOMATISÉ ET INDUSTRIALISÉ



Introduction et présentation du jeu de données	03
Modélisation et résultats	06
Présentation du pipeline de déploiement	22
Présentation de l'analyse de data drift	26
Exemple d'un scoring client via appel à l'API	29
Conclusion	31



Introduction et présentation du jeu de données



Contexte : Mise en place d'un outil de scoring crédit pour automatiser l'octroi de prêts à des clients sans historique bancaire.



Problématique : Le risque de défaut doit être anticipé pour éviter des pertes financières, tout en ne refusant pas trop de bons clients.



Objectifs : Développer un modèle de classification pour prédire la probabilité de remboursement et décider automatiquement accord/refus

Comment prédire efficacement le risque de défaut pour accorder ou refuser un crédit de façon automatisée ?

Présentation du jeu de données



Jeu de données

Données issues du challenge Kaggle **Home Credit Default Risk**

Dataset : Home Credit Default Risk (Kaggle)

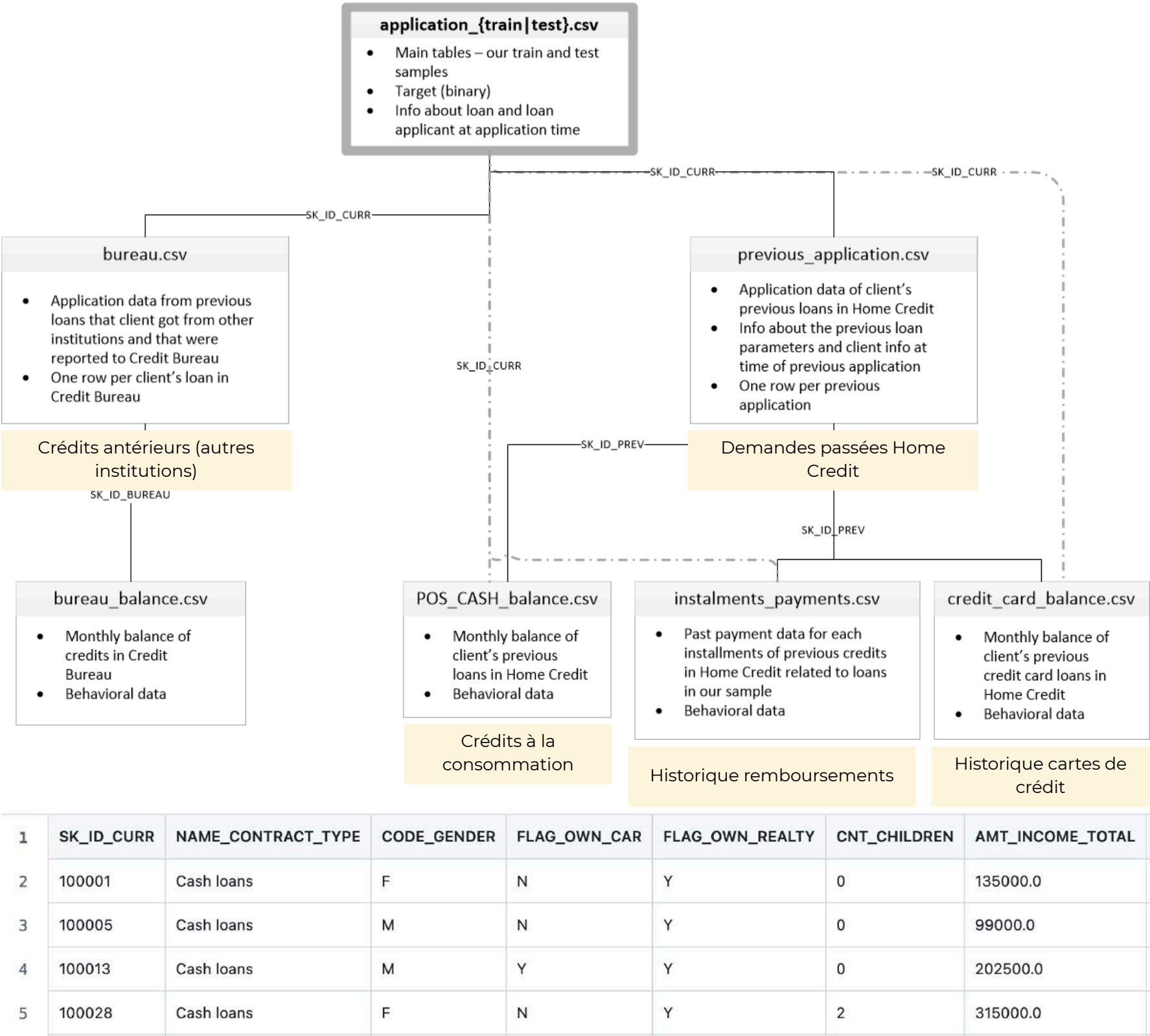
- 307,511 clients
- 7 tables relationnelles à joindre
- 122 variables initiales

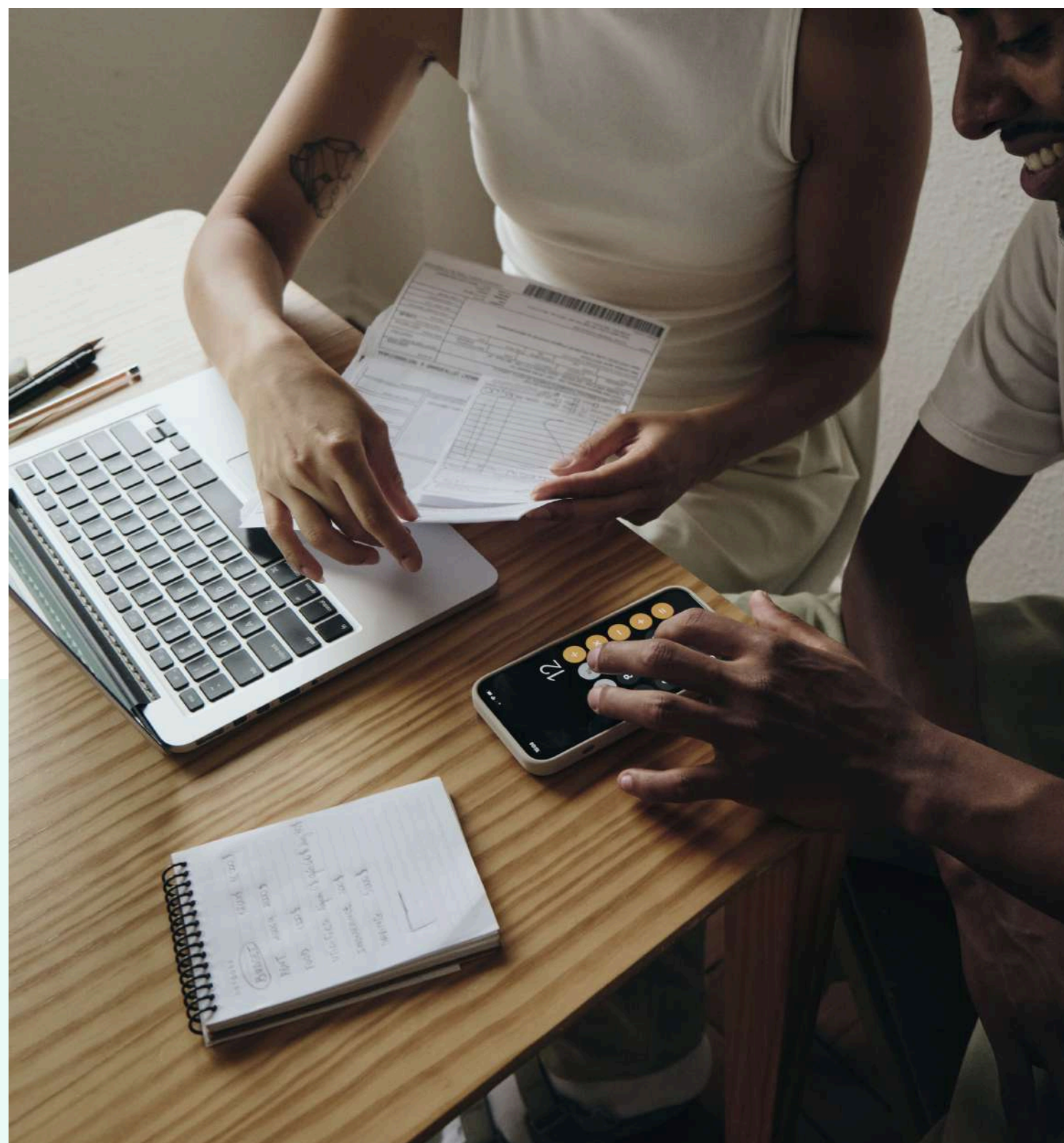
Déséquilibre des classes :

92% bons clients et 8% défauts

Asymétrie des coûts métiers :

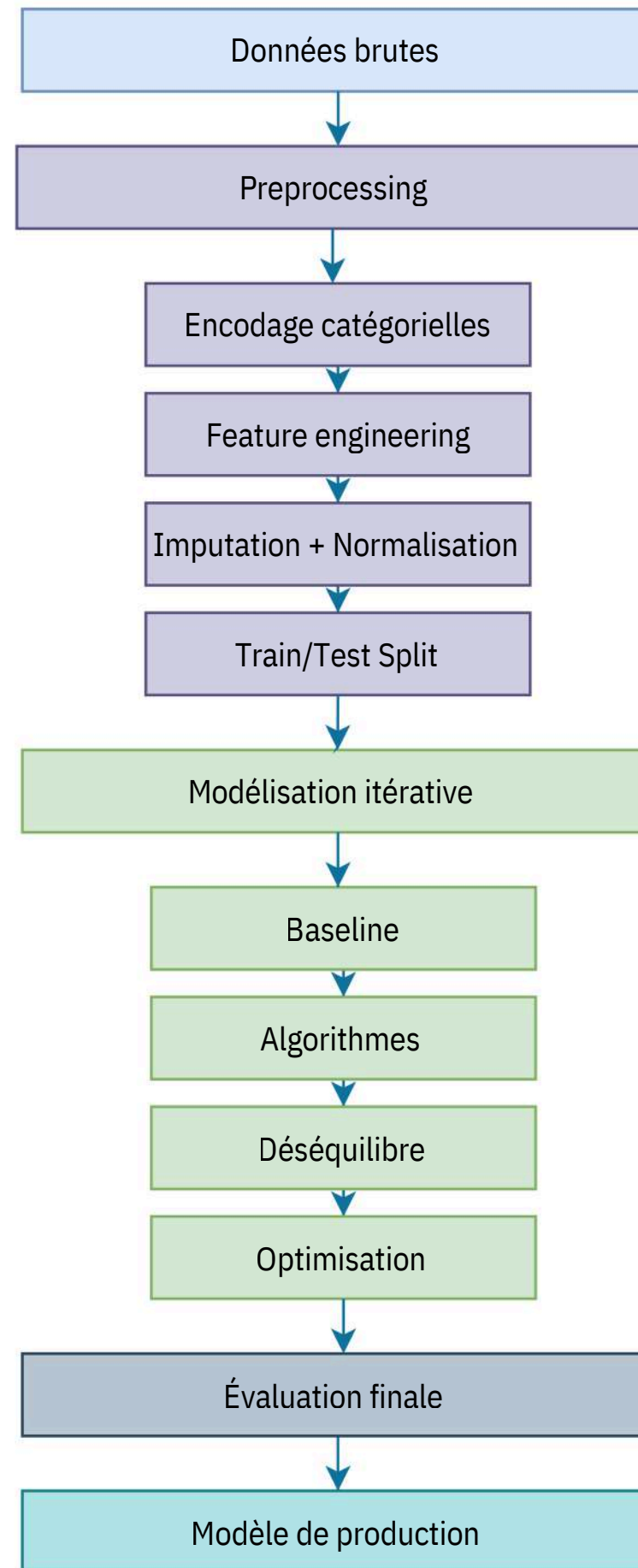
Coût faux négatif = 10 x coût faux positif
(perte capitale vs manque à gagner)



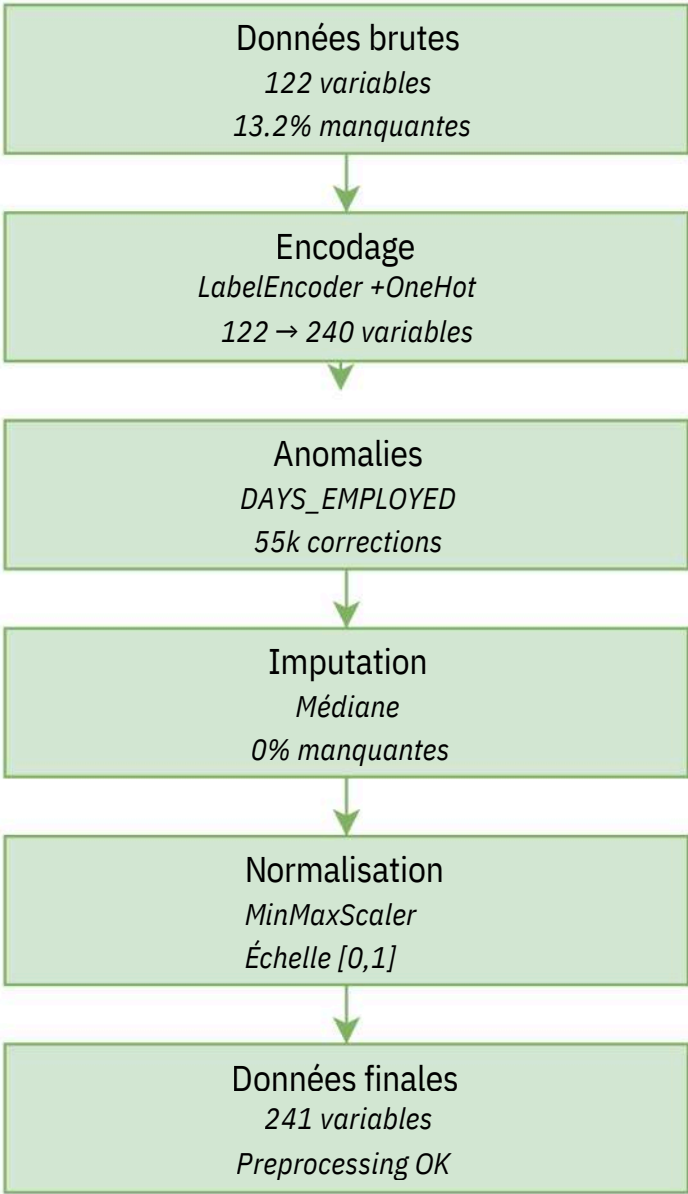


Modélisation et résultats

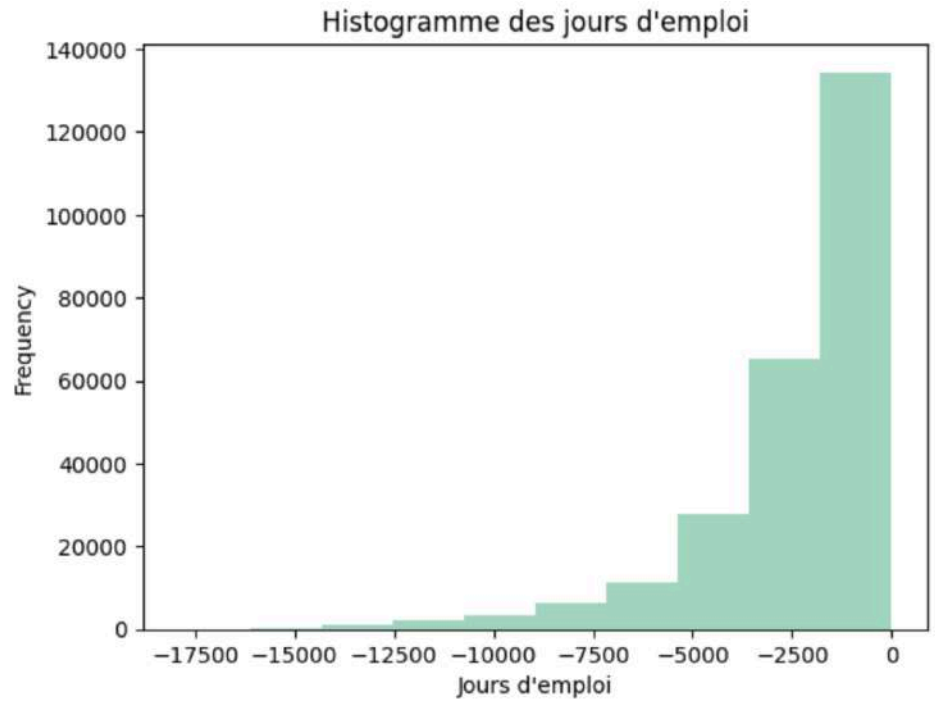
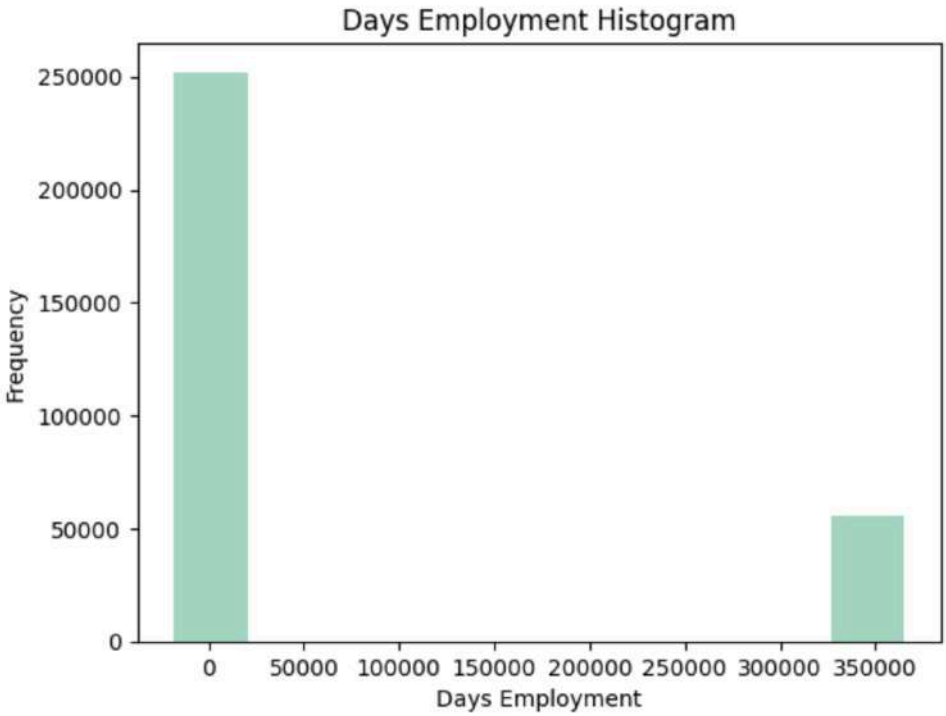
Approche de modélisation



Préparation des données



Validation croisée 10-fold pour robustesse



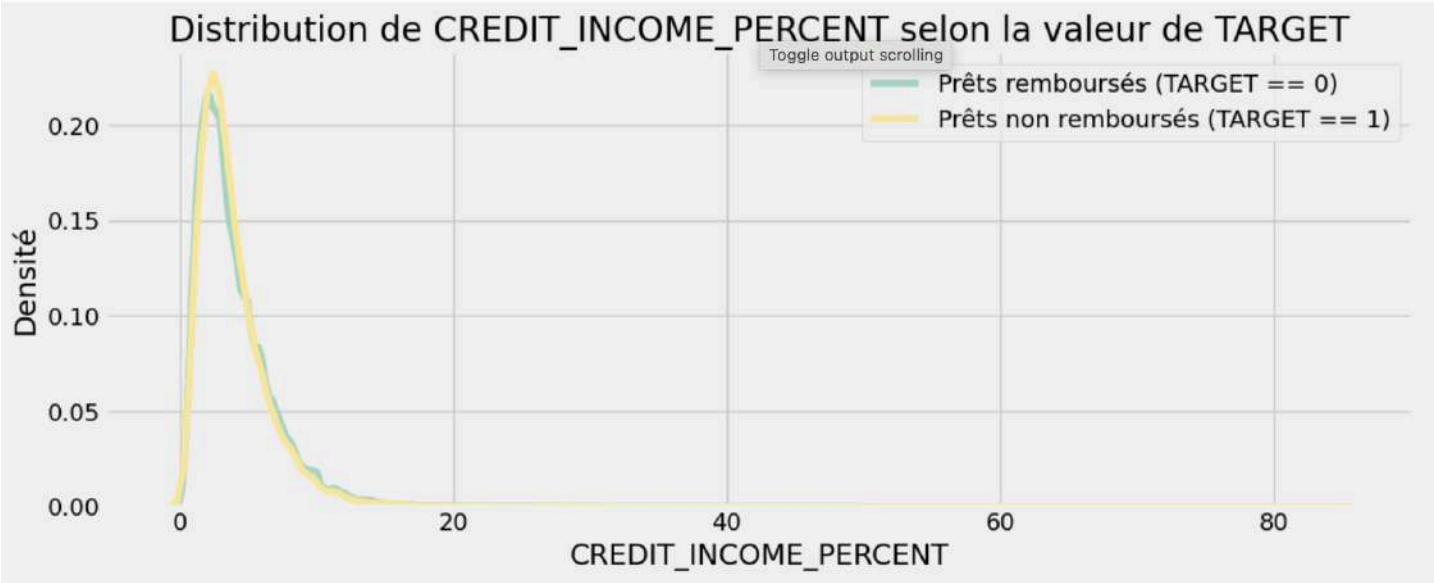
Variables polynomiales & interactions

- EXT_SOURCE combinées : (1×2), (1×3), (2×3)
- EXT_SOURCE × DAYS_BIRTH (âge × score externe)

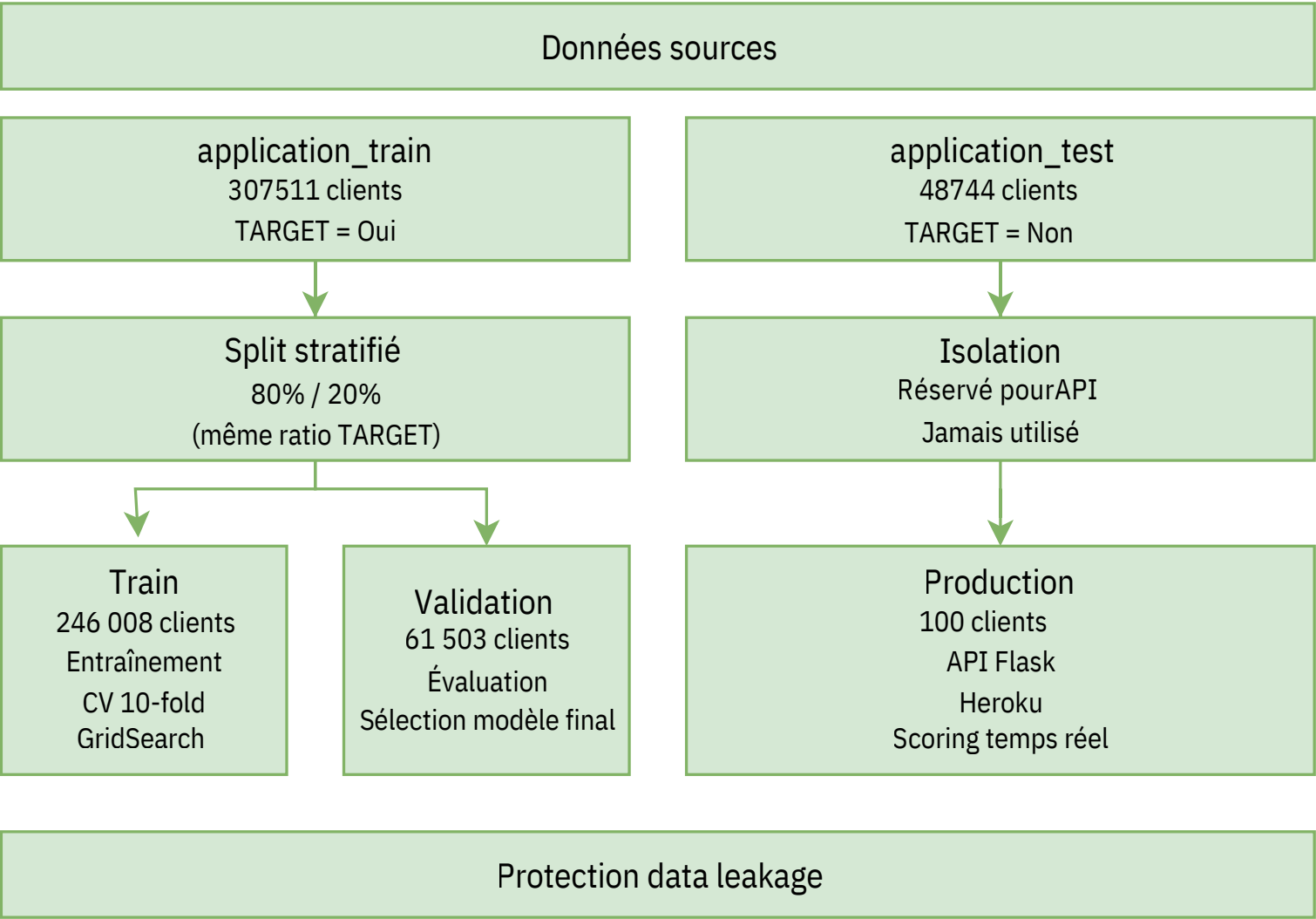
Variables métier créées

- CREDIT_INCOME_PERCENT : Taux d'endettement
- ANNUITY_INCOME_PERCENT : Effort de remboursement
- CREDIT_TERM : Durée estimée du crédit
- DAYS_EMPLOYED_PERCENT : Ancienneté relative

Impact : +15 variables créées



Stratégie de validation

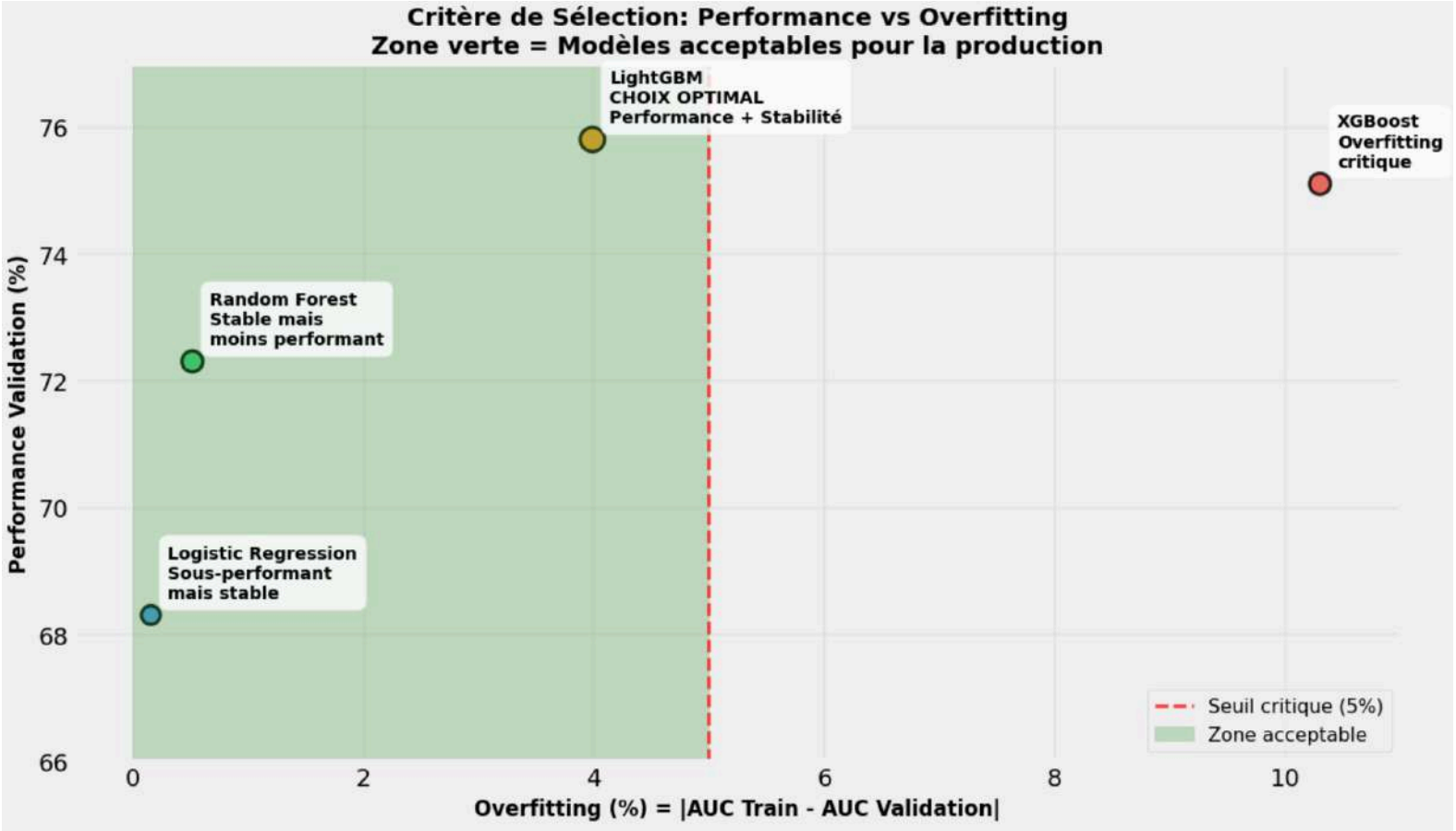


5 principes de validation

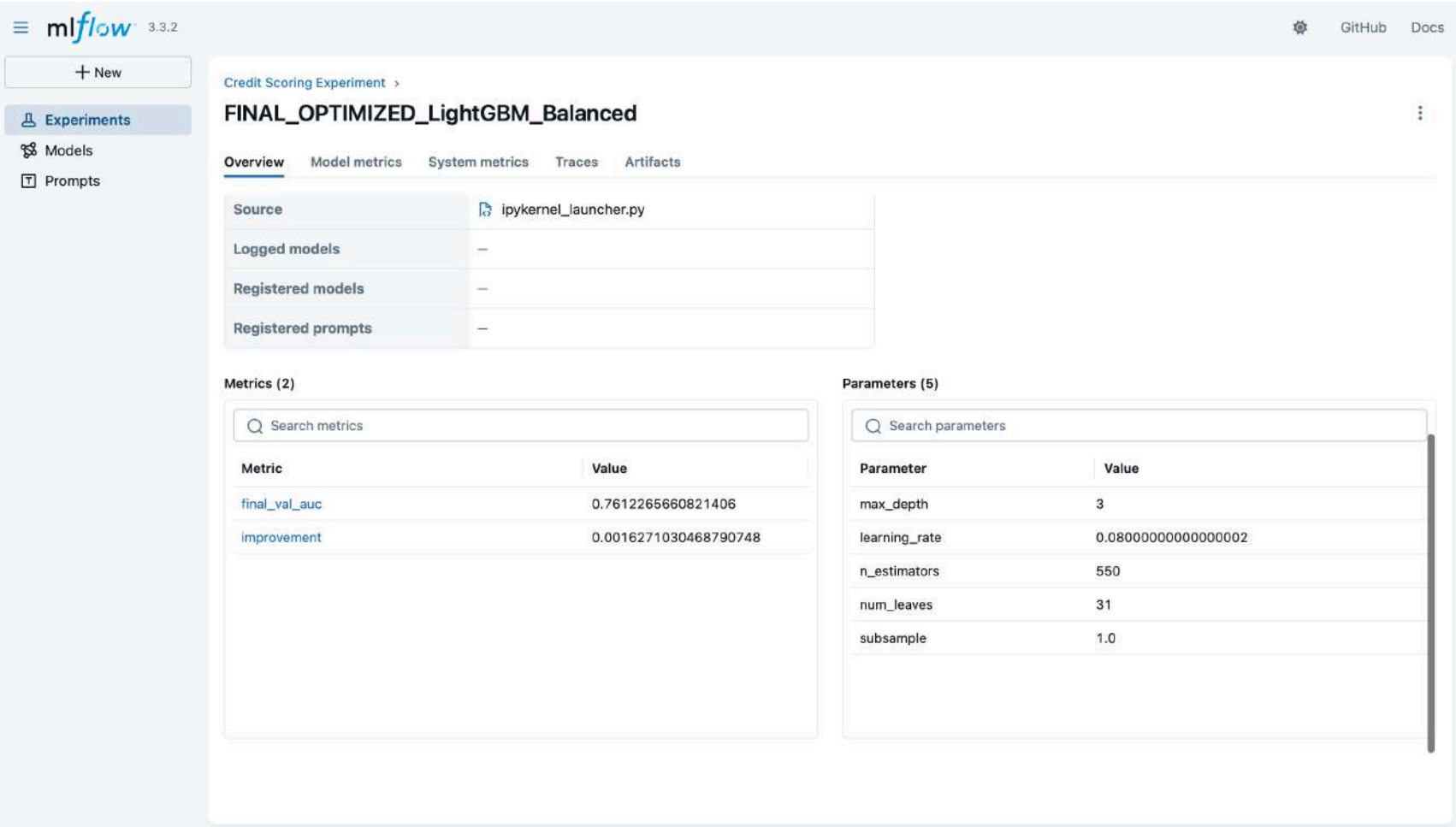
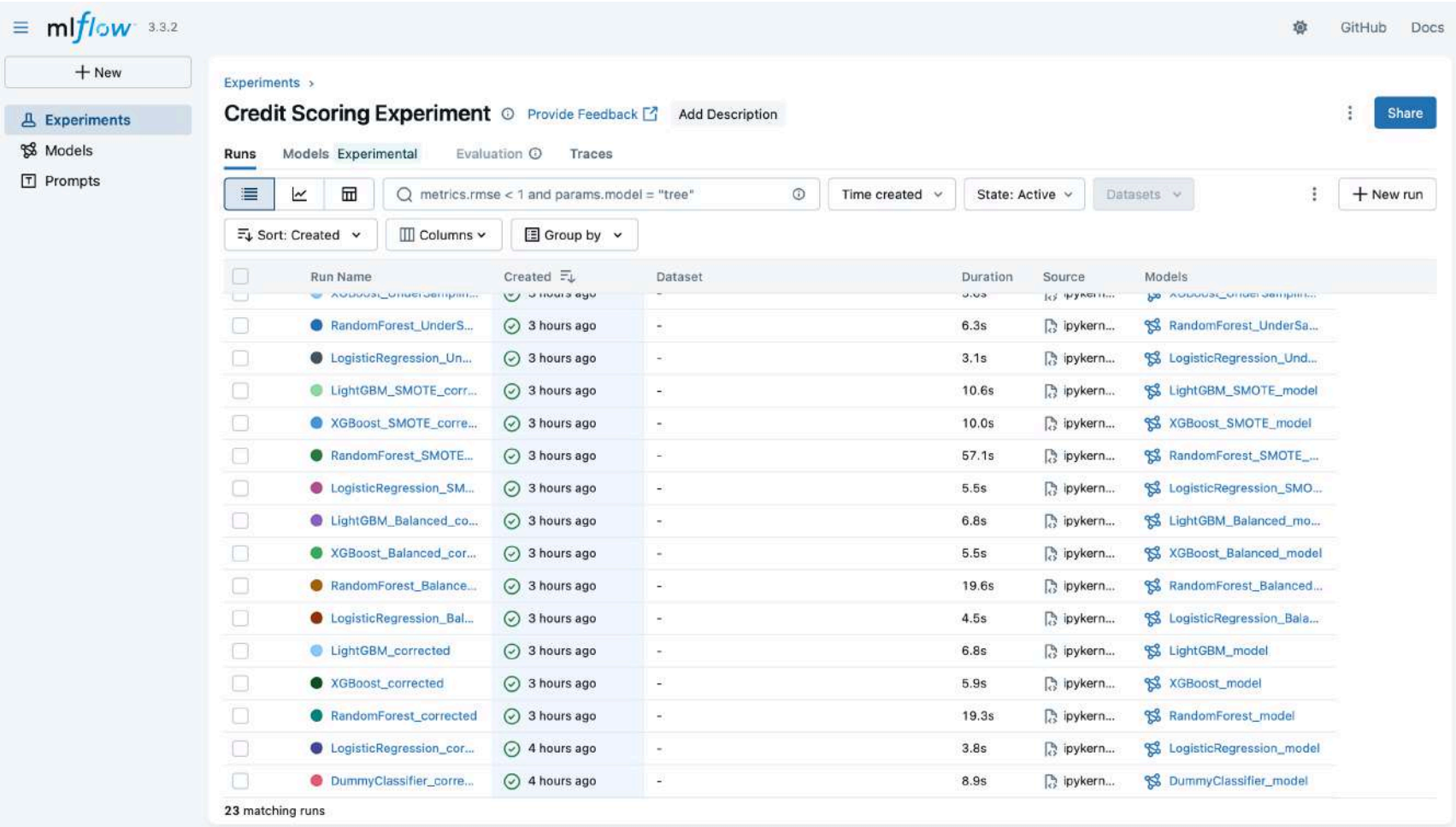
- Split AVANT preprocessing
- application_test isolé (simulation production)
- Cross-validation sur train uniquement
- Validation set : sélection finale seulement
- Zéro fuite d'information entre datasets

Modèles testés et baseline

Modèle	Train AUC	Validation AUC	Overfitting Gap
DummyClassifier	0.500	0.500	0.000
Logistic Regression	0.681	0.683	-0.0016
Random Forest	0.728	0.723	0.0052
XGBoost	0.854	0.751	0.1030
LightGBM	0.798	0.758	0.0399

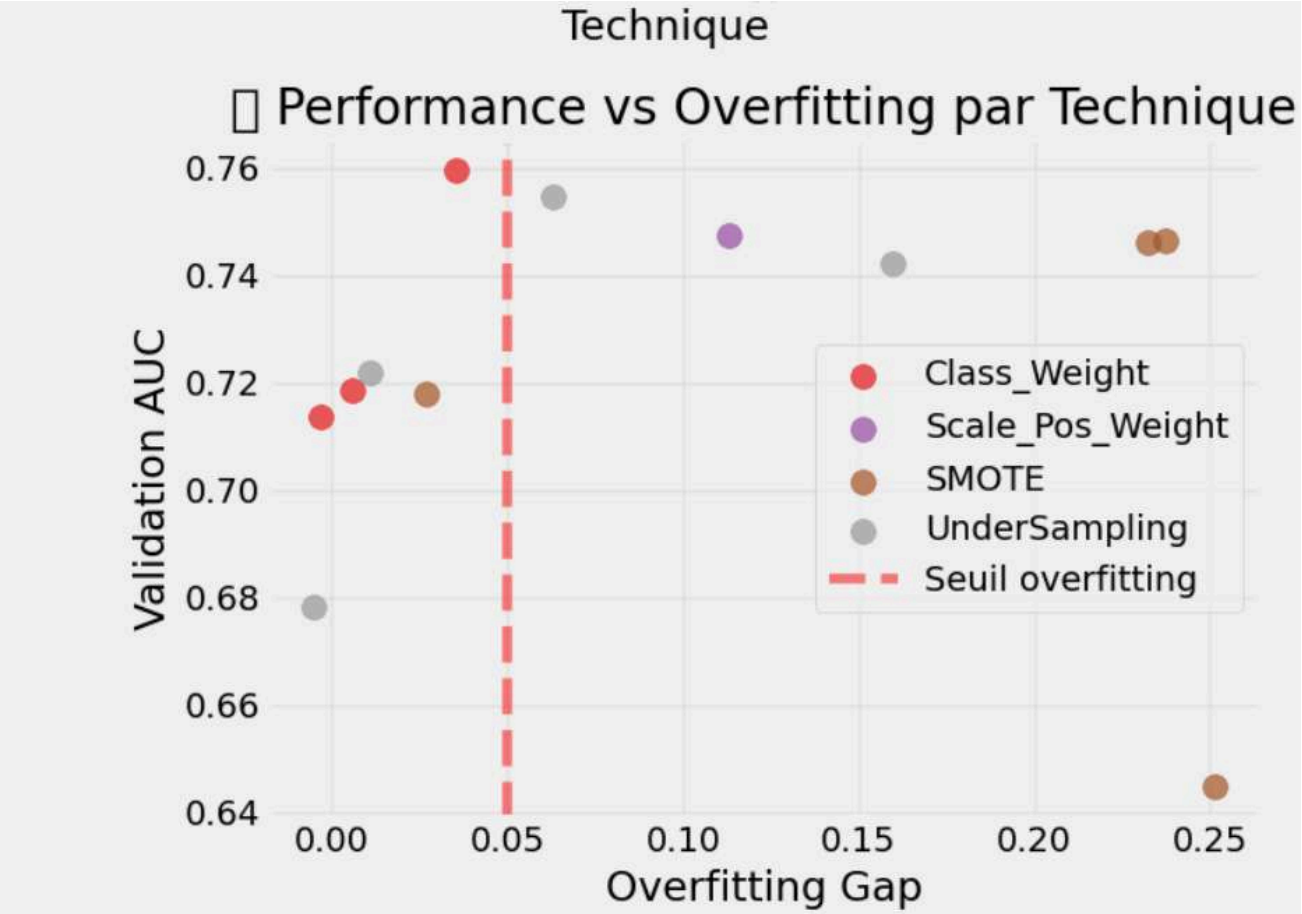


Suivi MLflow - Tracking



- Tracking de 15+ expérimentations
- Comparaison automatique des métriques
- Stockage centralisé des modèles
- Historique complet des hyperparamètres

Modèle	Baseline	Class Weight	SMOTE	UnderSampling	Meilleure méthode	Gain
Logistic Regression	0.683	0.714	0.718	0.678	SMOTE	+0.035
Random Forest	0.723	0.719	0.645	0.722	UnderSampling	-0.001
XGBoost	0.751	0.747	0.746	0.742	Scale_Pos_Weight	-0.004
LightGBM	0.758	0.760	0.746	0.755	Class Weight	+0.002



Modèle sélectionné : LightGBM + Class Weight Balanced
Objectif : Optimiser au-delà de l'AUC baseline (0.758)

Approche en 2 phases

RandomizedSearch

- **Principe** : Échantillonnage aléatoire de 50 combinaisons
- **Exploration** : 5 types de paramètres, 3-fold CV
- **Temps** : 507.1 secondes

GridSearch

- **Principe** : Test exhaustif de 27 combinaisons prédéfinies
- **Exploration** : Affinement autour des meilleurs paramètres RandomizedSearch
- **Temps** : 528.7 secondes (+4% vs RandomizedSearch)

Modèle	CV AUC	Val AUC	Gain
Baseline	0.758		
RandomizedSearch	0.7546	0.7619	+0.0073
GridSearch Final	0.7567	0.7612	+0.0045

Paramètres optimaux: {'subsample': 1.0, 'num_leaves': 31, 'learning_rate': 0.080000000000000002, 'max_depth': 3, 'n_estimators': 550}

Suivi MLflow - Comparaison

mlflow

3.3.2

GitHub

Docs

+ New

Experiments

Models

Prompts

Credit Scoring Experiment >

Comparing 2 Runs from 1 Experiment

Run details

Run ID:	dbee5cd1c1914fb2a2a172ff8abd52ce	758717c6e1a742918acba47a8d359a1
Run Name:	RandomizedSearch_LightGBM_Balanced	GridSearch_LightGBM_Balanced
Start Time:	09/28/2025, 11:34:36 AM	09/28/2025, 11:43:08 AM
End Time:	09/28/2025, 11:43:08 AM	09/28/2025, 11:52:01 AM
Duration:	8.5min	8.9min

Parameters

Show diff only

learning_rate	0.1	0.080000000000000002
max_depth	3	3
n_estimators	500	550
num_leaves	31	31
subsample	1.0	1.0

mlflow

3.3.2

GitHub

Docs

+ New

Experiments

Models

Prompts

n_estimators	500	550
num_leaves	31	31
subsample	1.0	1.0

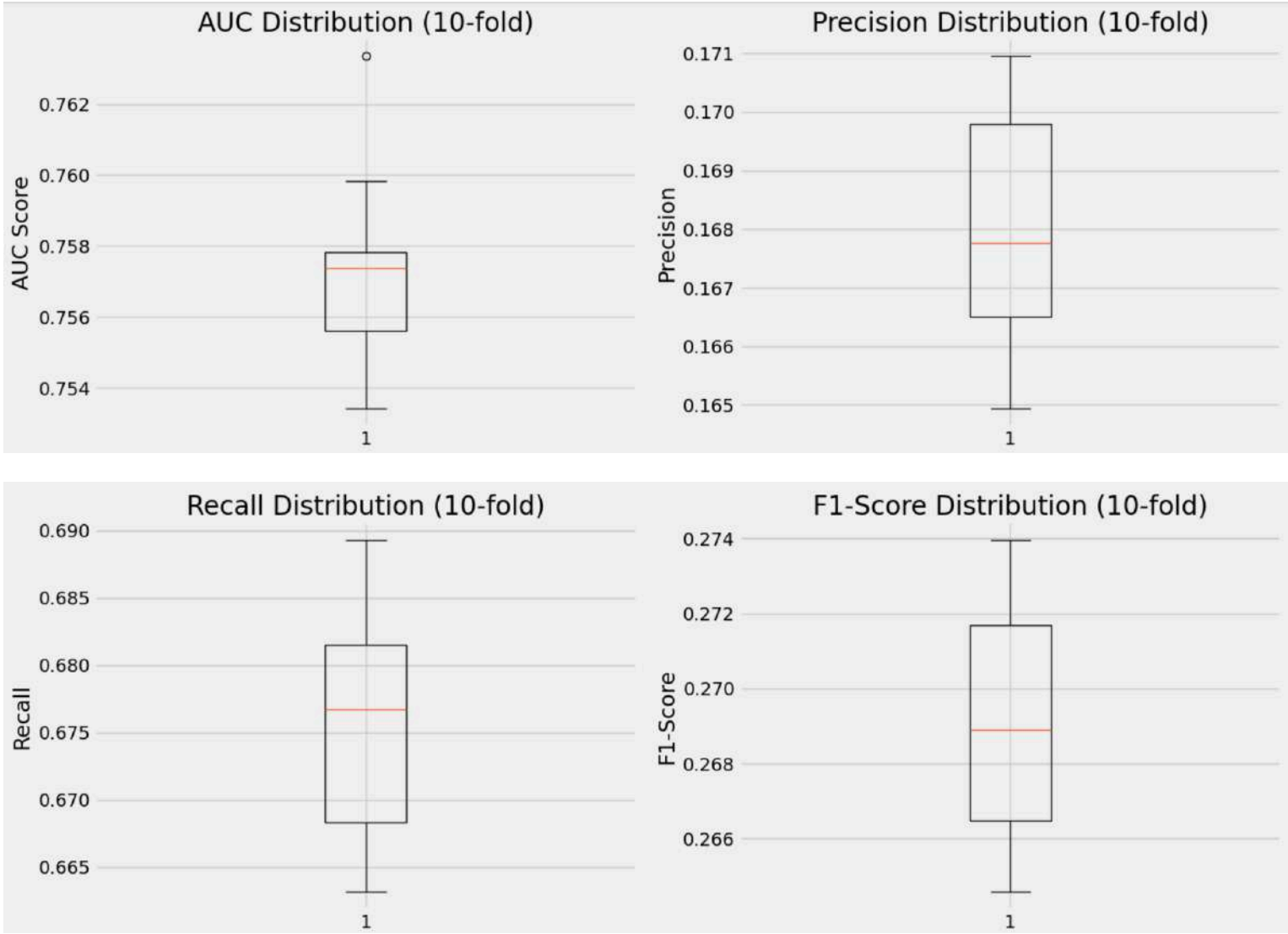
Metrics

Show diff only

best_cv_score	0.755	0.757
n_iterations	50	
search_time	507.1	528.7
val_auc	0.762	0.761

Validation croisée
robuste

Métrique	Score test	Écart-type	Gap Train-Test
AUC-ROC	75.7%	±0.3%	3.0%
Accuracy	70.4%	±0.2%	0.7%
Precision	16.8%	±0.2%	1.1%
Recall	67.5%	±0.9%	4.2%
F1-Score	26.9%	±0.3%	1.7%

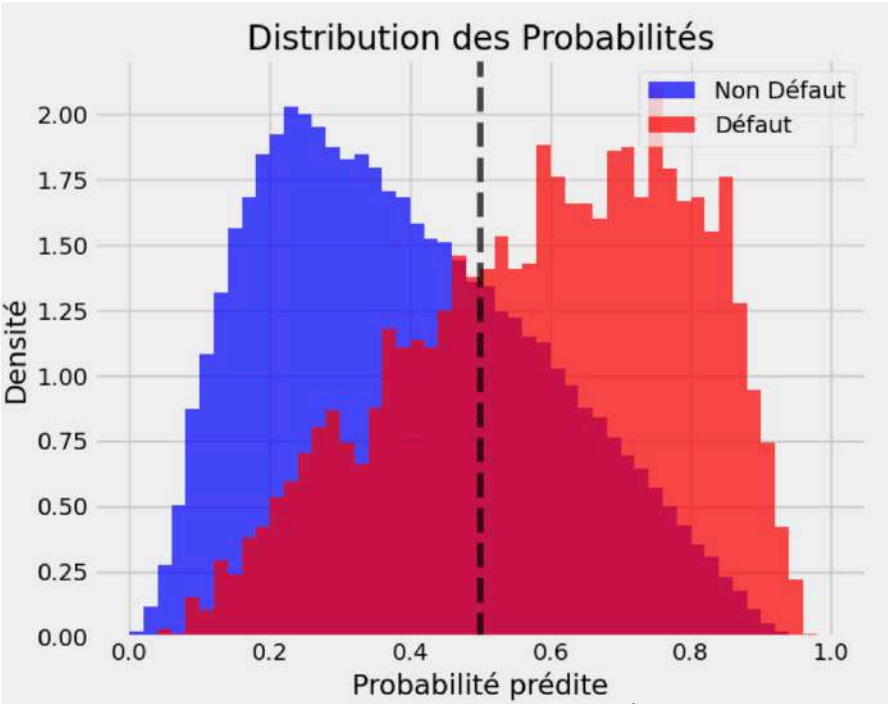


Modèle retenu : LightGBM avec hyperparamètres optimisés

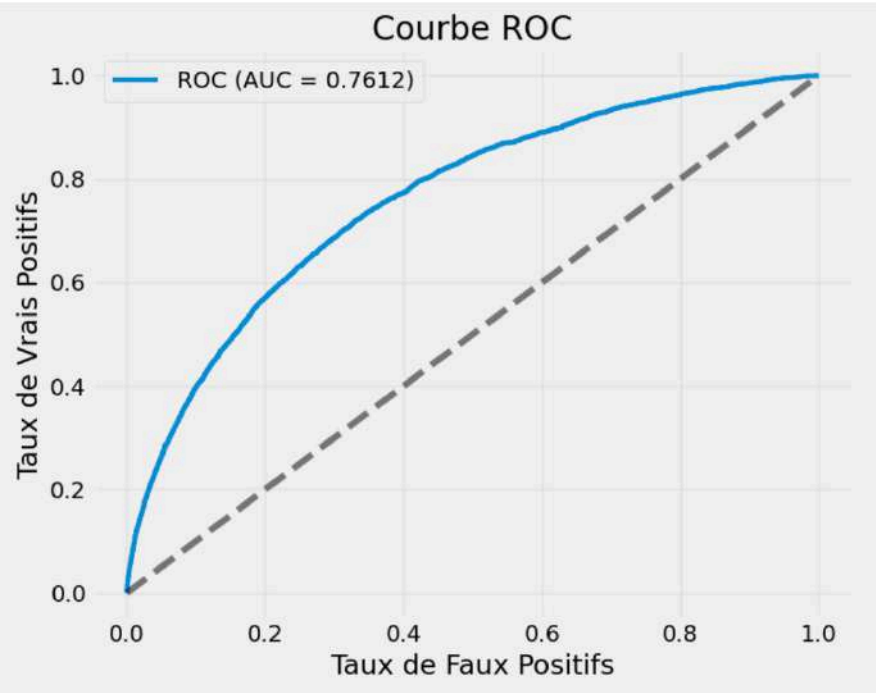
- AUC validation : 76.1% (train: 78.5%)
- Overfitting maîtrisé : gap = 2.4% < 5%
- Performance rapide : 3.7s train, 0.6s prédiction

Performance métier optimisée

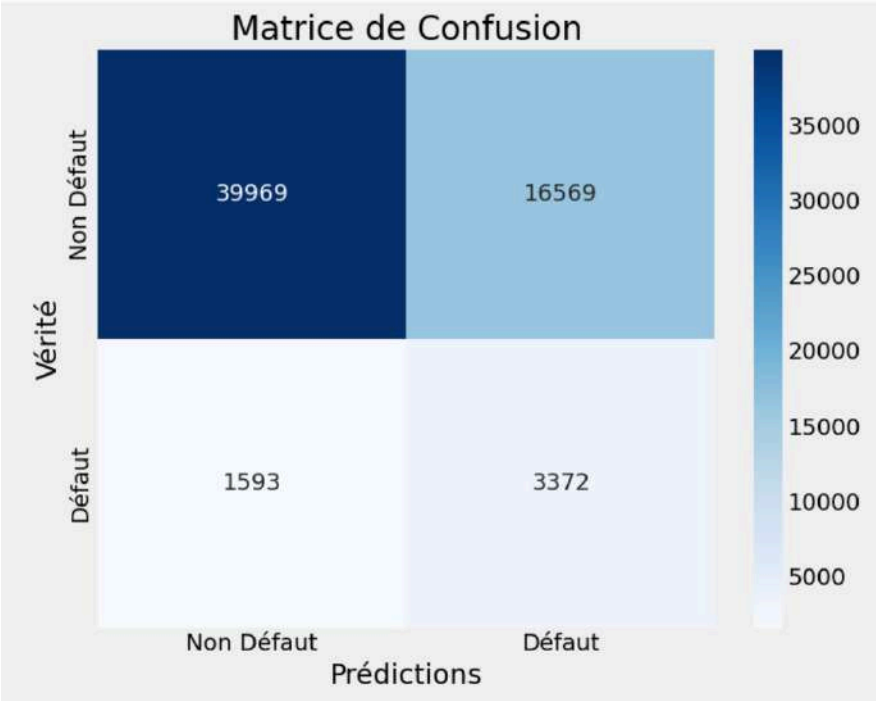
- Recall 67.9% : capture 2/3 des vrais défauts
- Precision 16.9% : 1 vrai défaut sur 6 alertes
- Accuracy 70.5% : classification globale satisfaisante



Distribution probabilités : séparation claire bon/mauvais clients

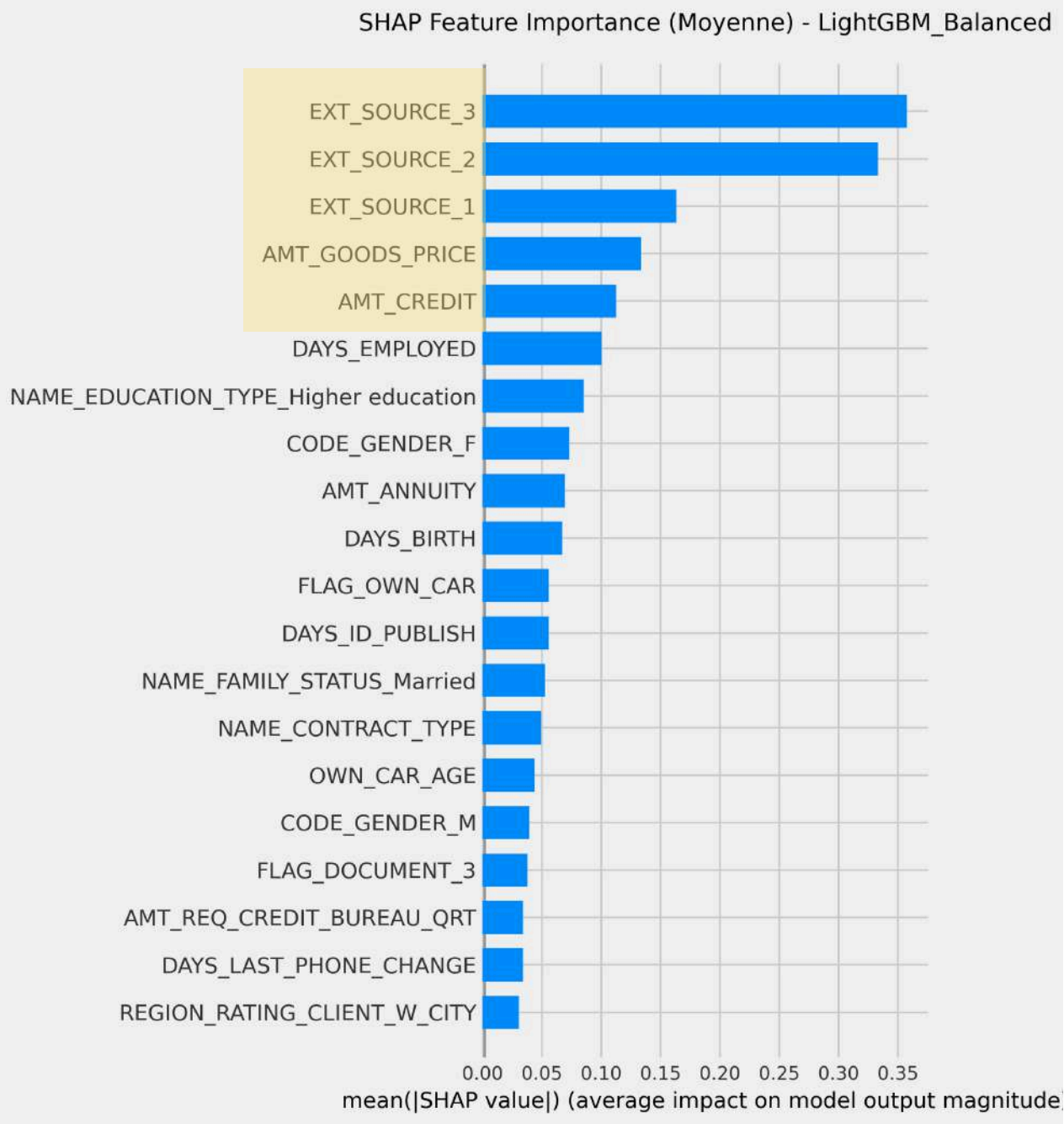
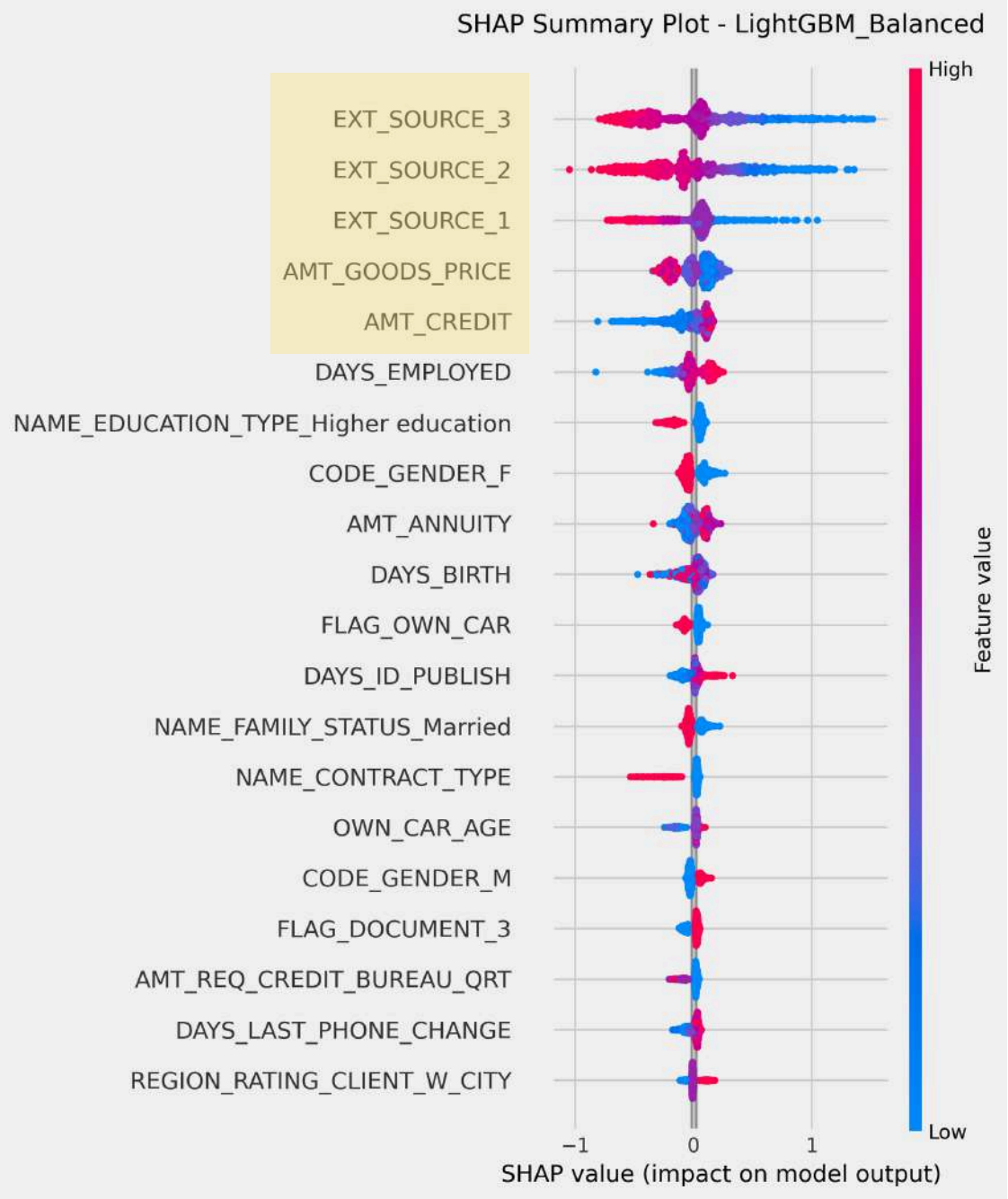


Courbe ROC : discrimination efficace des classes



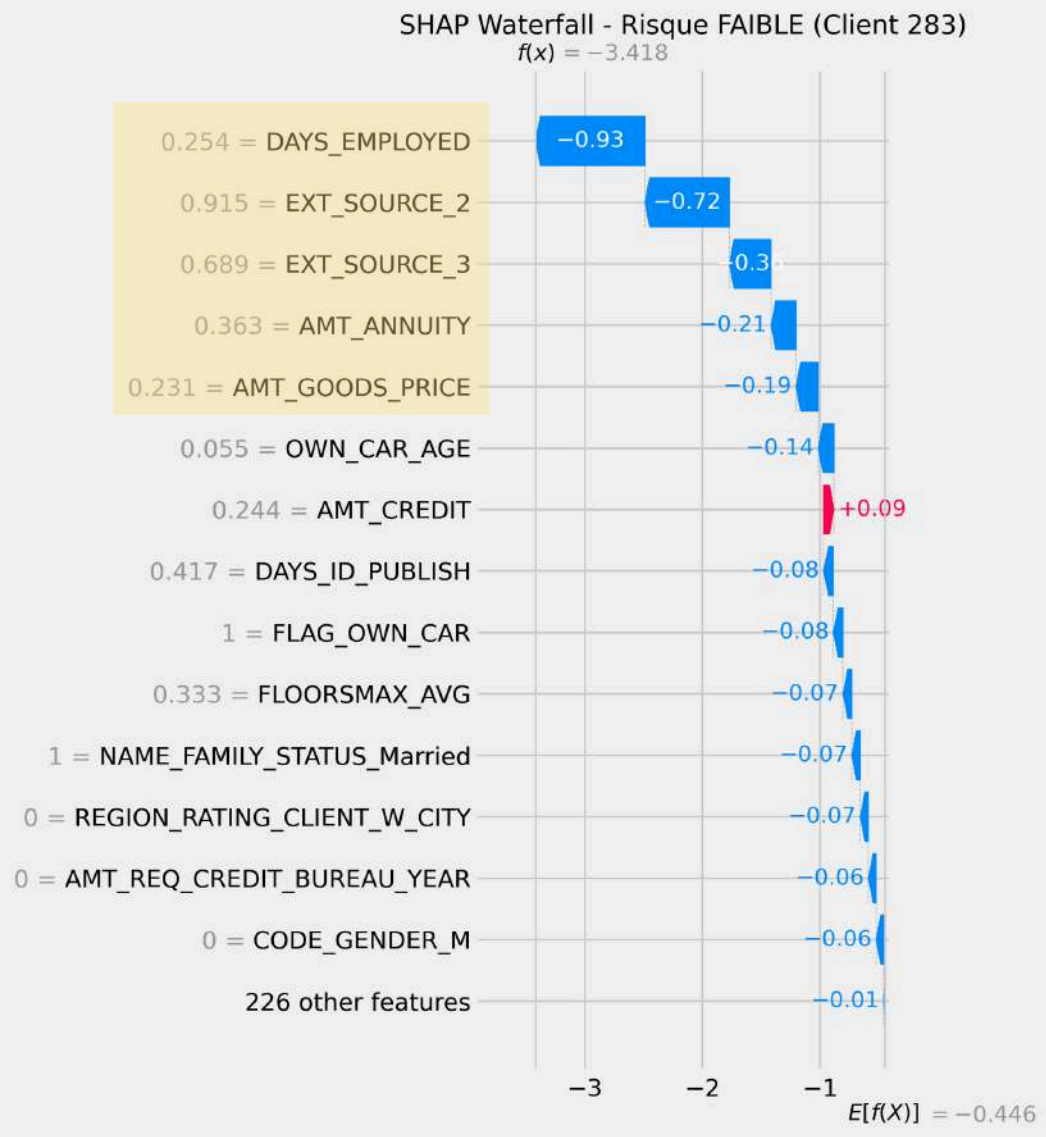
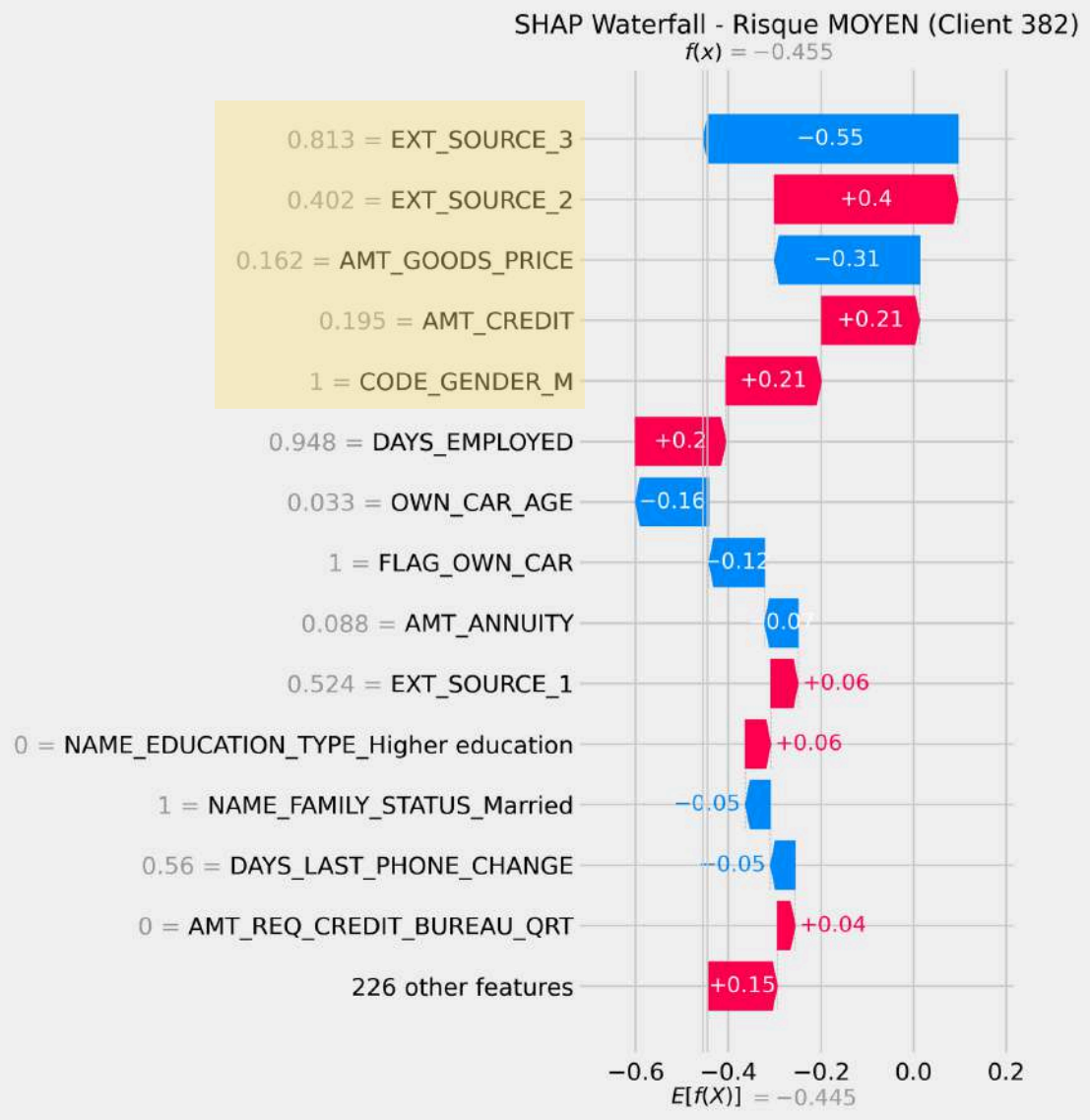
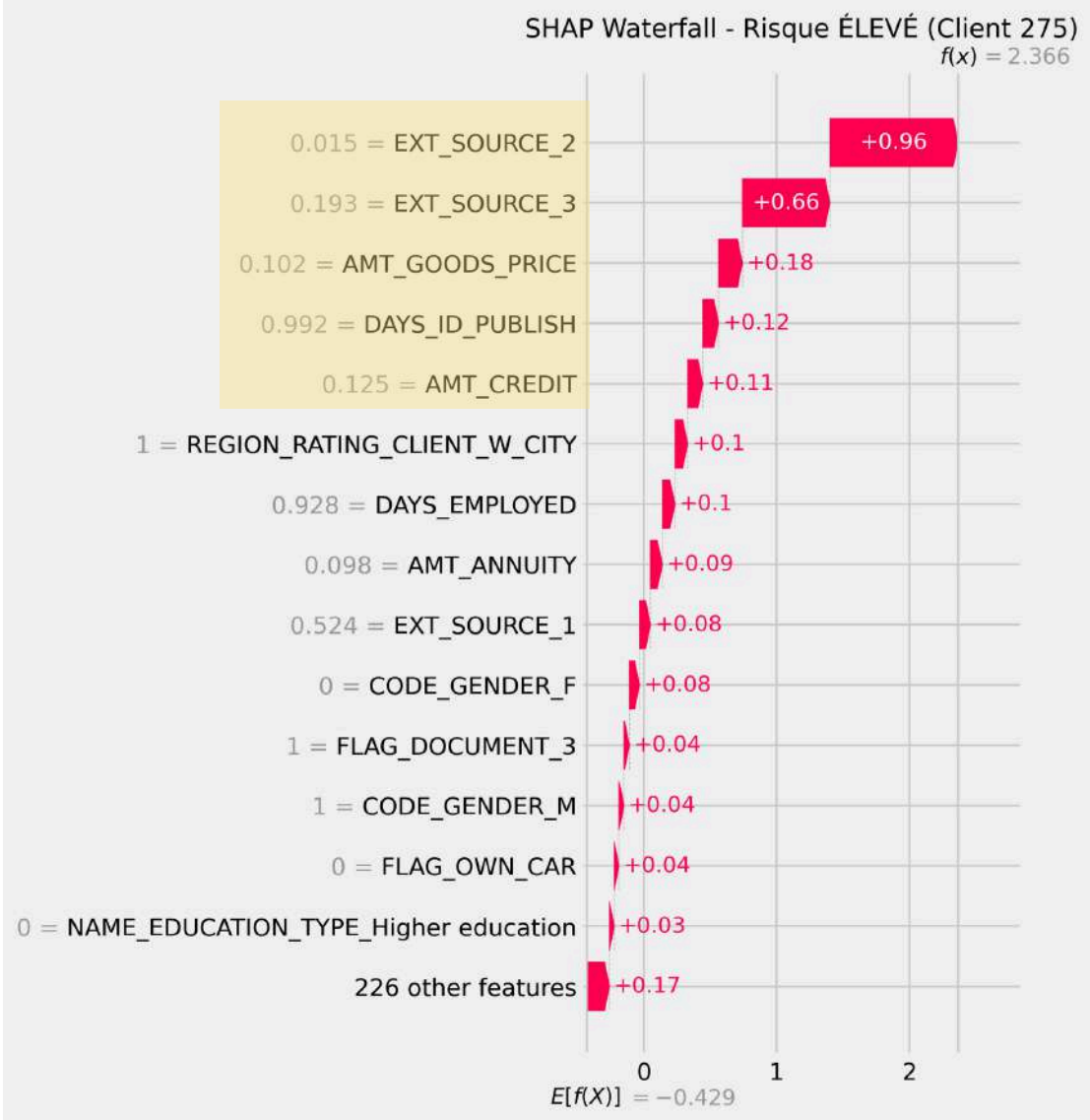
Matrice confusion : 3,372 TP / 1,593 FN détectés

Feature Importance Globale (SHAP)



Feature Importance

Locale (SHAP)



Optimisation seuil métier

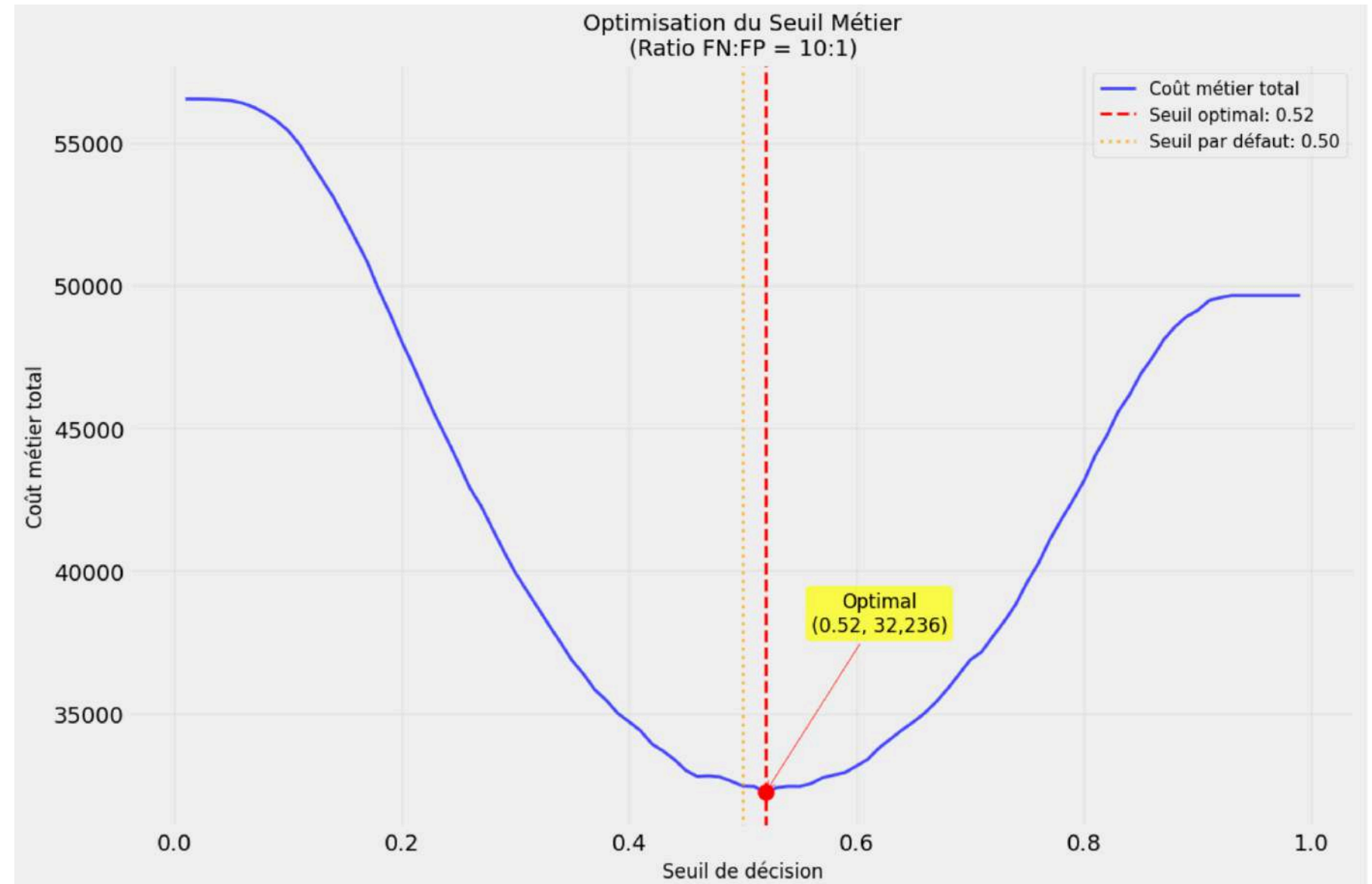
Quel seuil nous coûte le moins cher ?

Méthode "Force brute intelligente"

- Tester TOUS les seuils possibles (0.01, 0.02, 0.03... 0.99)
- Pour chaque seuil, calculer le coût métier total
- Sélection du seuil à coût minimal

Formule coût métier : $\text{Coût Total} = 10 \times \text{FN} + 1 \times \text{FP}$

- FN (Faux Négatif) : Mauvais client accepté → Perte capitale
- FP (Faux Positif) : Bon client refusé → Manque à gagner
- Ratio 10:1 reflète l'impact financier réel



Seuil 0.50 (défaut) :

- 1,593 fraudes manquées × 10€ = 15,930€
- 16,569 clients bloqués × 1€ = 16,569€

→ COÛT TOTAL = 32,499€ (+115€ vs optimal)

Seuil 0.52 :

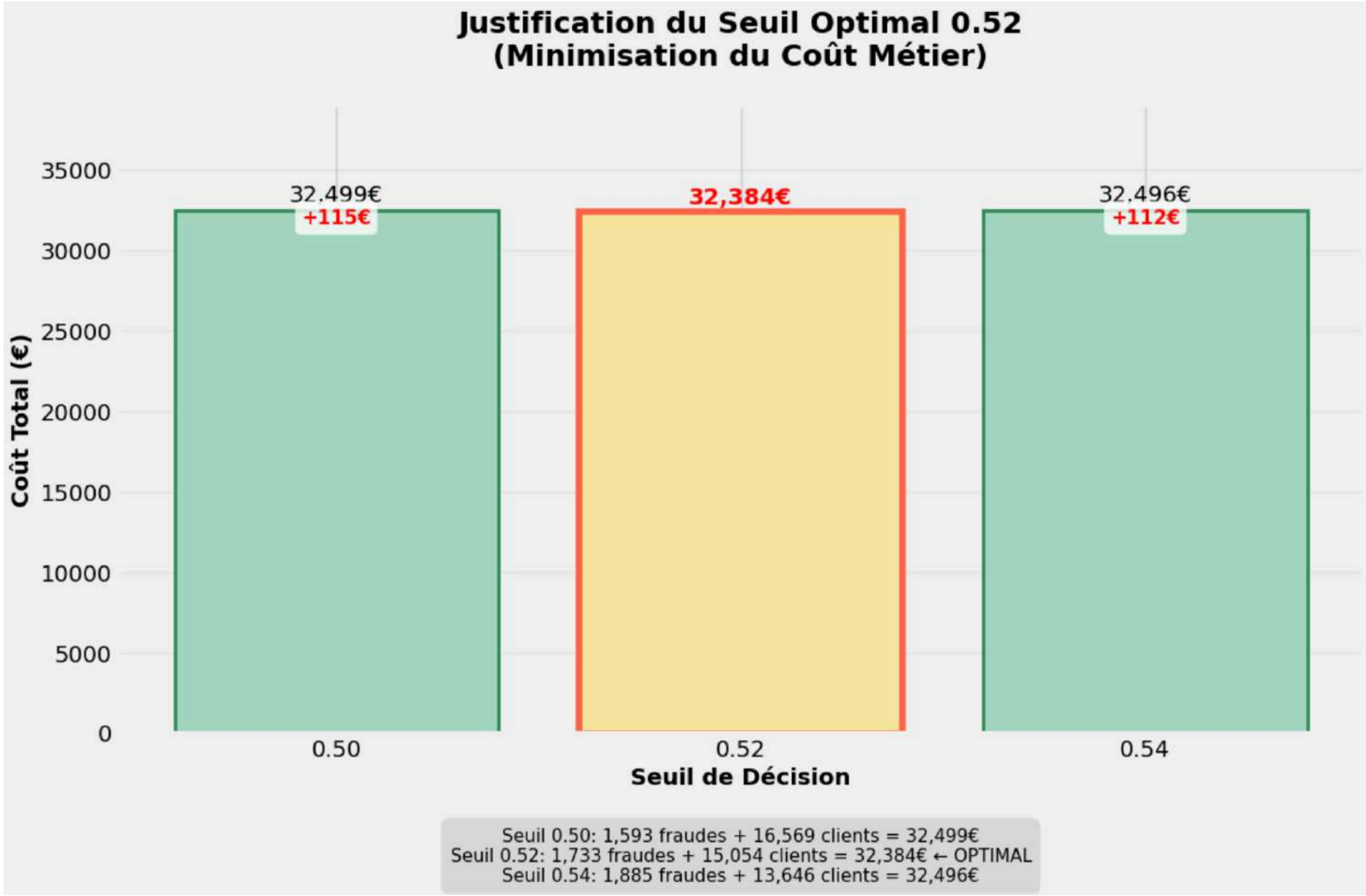
- 1,733 fraudes manquées × 10€ = 17,330€
- 15,054 clients bloqués × 1€ = 15,054€

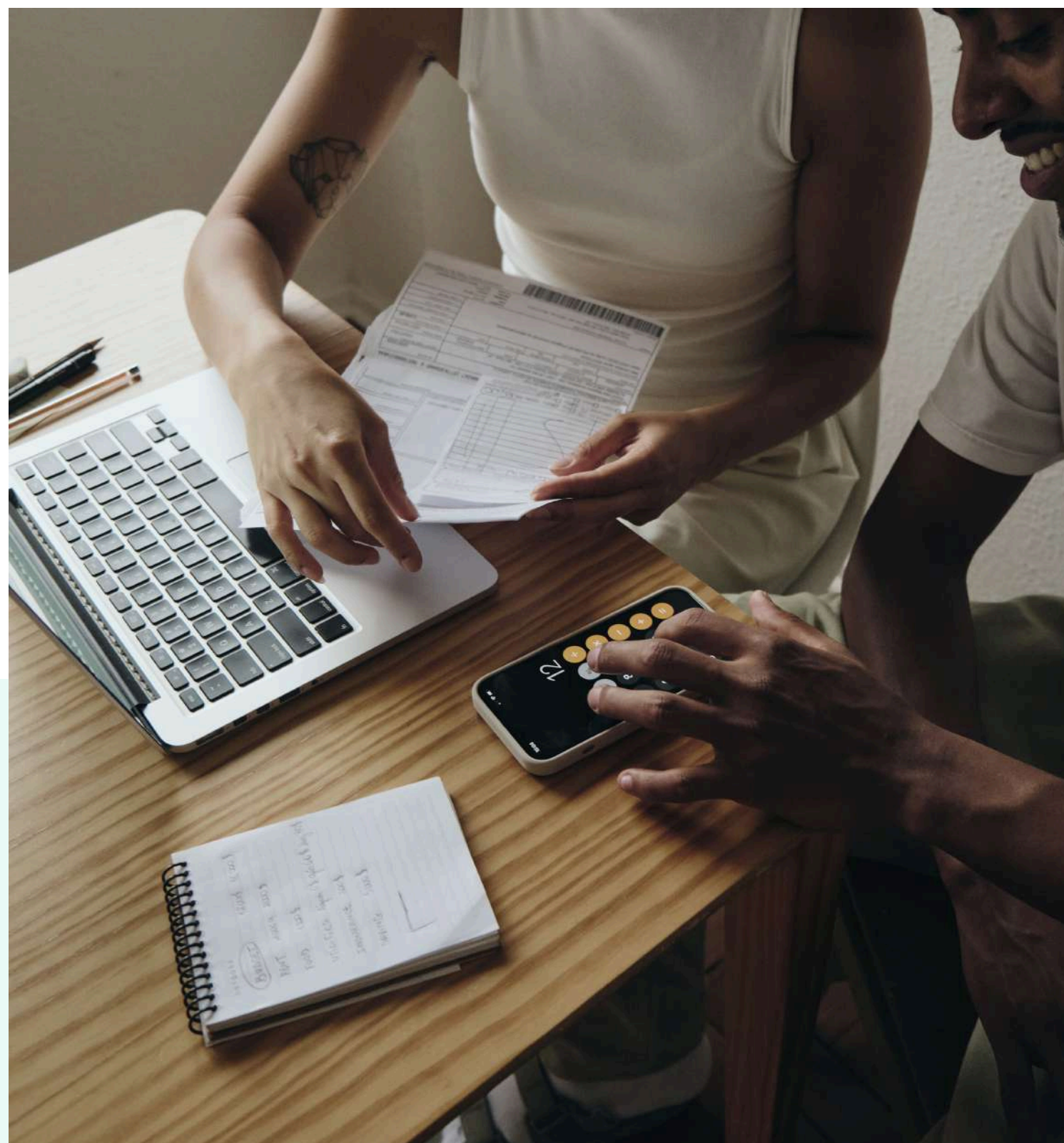
→ COÛT TOTAL = 32,384€ (minimum)

Seuil 0.54 :

- 1,885 fraudes manquées × 10€ = 18,850€
- 13,646 clients bloqués × 1€ = 13,646€

→ COÛT TOTAL = 32,496€ (+112€ vs optimal)





Présentation du pipeline de déploiement

Versioning & Collaboration

▼

.github/workflows

ci.yml

▼

app

README.md

api.py

application_test.csv

features.pkl

imputer.pkl

model_complet.pkl

requirements.txt

scaler.pkl

▼

notebooks

notebook_data_drift_evidently.ipynb

notebook_modélisation.ipynb

▼

tableau_html

tableau_HTML_data_drift_evidently.h...

▼

tests

test_api.py

test_preprocess.py

.DS_Store

.gitignore

.python-version

LICENSE

Procfile

README.md

api.py

app_streamlit.py

jademayalb / credit-scoring-api

Type ↵ to search

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

Actions

New workflow

All workflows

CI

Management

Caches

Deployments

Attestations

Runners

Usage metrics

Performance metrics

All workflows

Showing runs from all workflows

is:success

14 workflow run results

Event	Status	Branch	Actor
Ajout README global, requirements_dev.txt et ap...	success	main	jademayalb
Ajout explicite des fichiers modèles dans app/	success	main	jademayalb
Déplace api.py à la racine pour Heroku	success	main	jademayalb
Corrige Procfile pour api.py	success	main	jademayalb
Ajout de gunicorn pour Heroku	success	main	jademayalb
Déplacement du Procfile à la racine pour Heroku	success	main	jademayalb

jademayalb / credit-scoring-api

Type ↵ to search

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

Releases Tags

Tags

v4

last week

2911513

zip

tar.gz

v3

last week

671aaaa

zip

tar.gz

v2

last week

dbd8b9c

zip

tar.gz

v1

2 weeks ago

9458a5e

zip

tar.gz

Versioning complet du code

app/ : Code source principal de l'API et des fonctionnalités.

tests/ : Scripts de tests automatisés pour assurer la qualité.

notebooks/ : Analyses exploratoires, prototypage et visualisation des données.

Workflow CI/CD avec GitHub Actions

Intégration continue : tests automatiques à chaque push/pull request.

Déploiement continu : automatisation du déploiement de l'API.

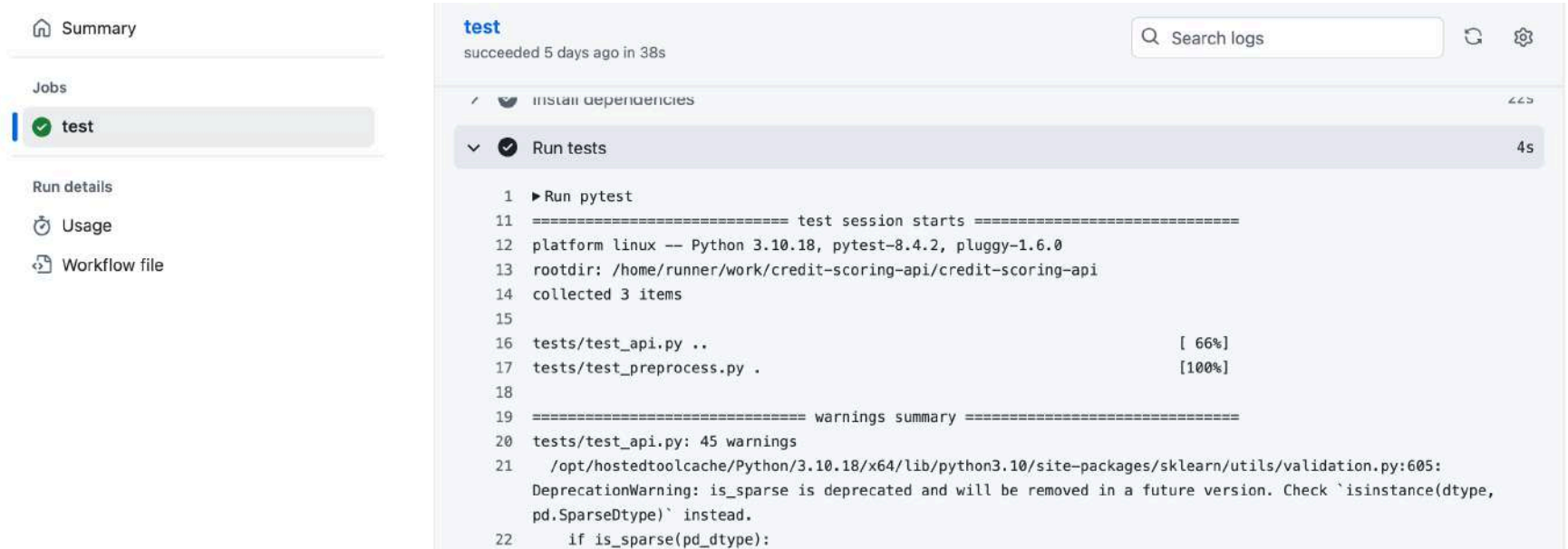
Tests unitaires avec pytest :

test_preprocess.py

Validation format des données
Vérification colonnes attendues
Contrôle valeurs manquantes

test_api.py

Test endpoint /predict/{id}
Validation codes de retour (200/404)
Vérification structure réponse JSON



Code Push → pytest → Quality Check → Deploy

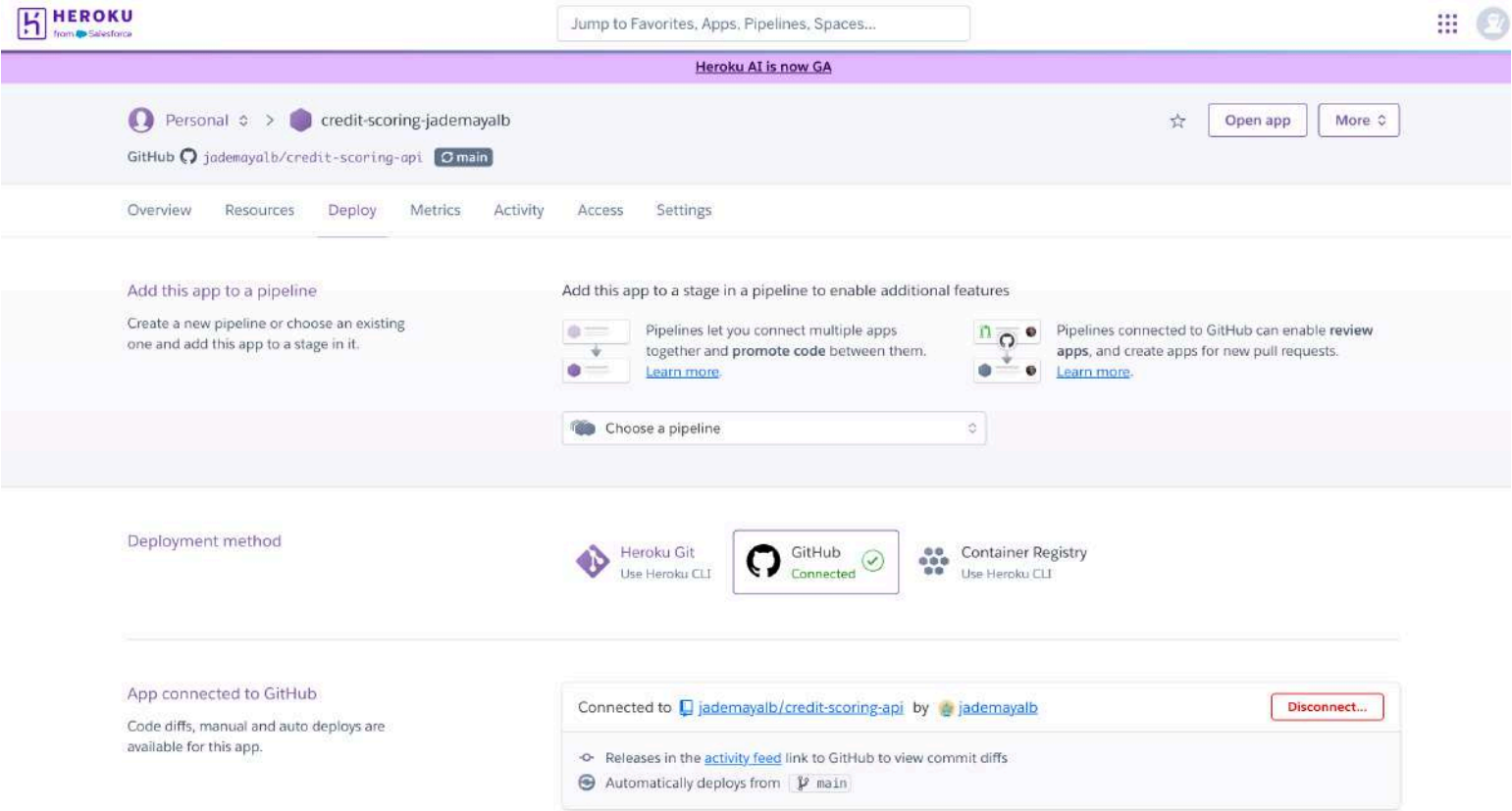
Objectif : Garantir la qualité avant mise en production

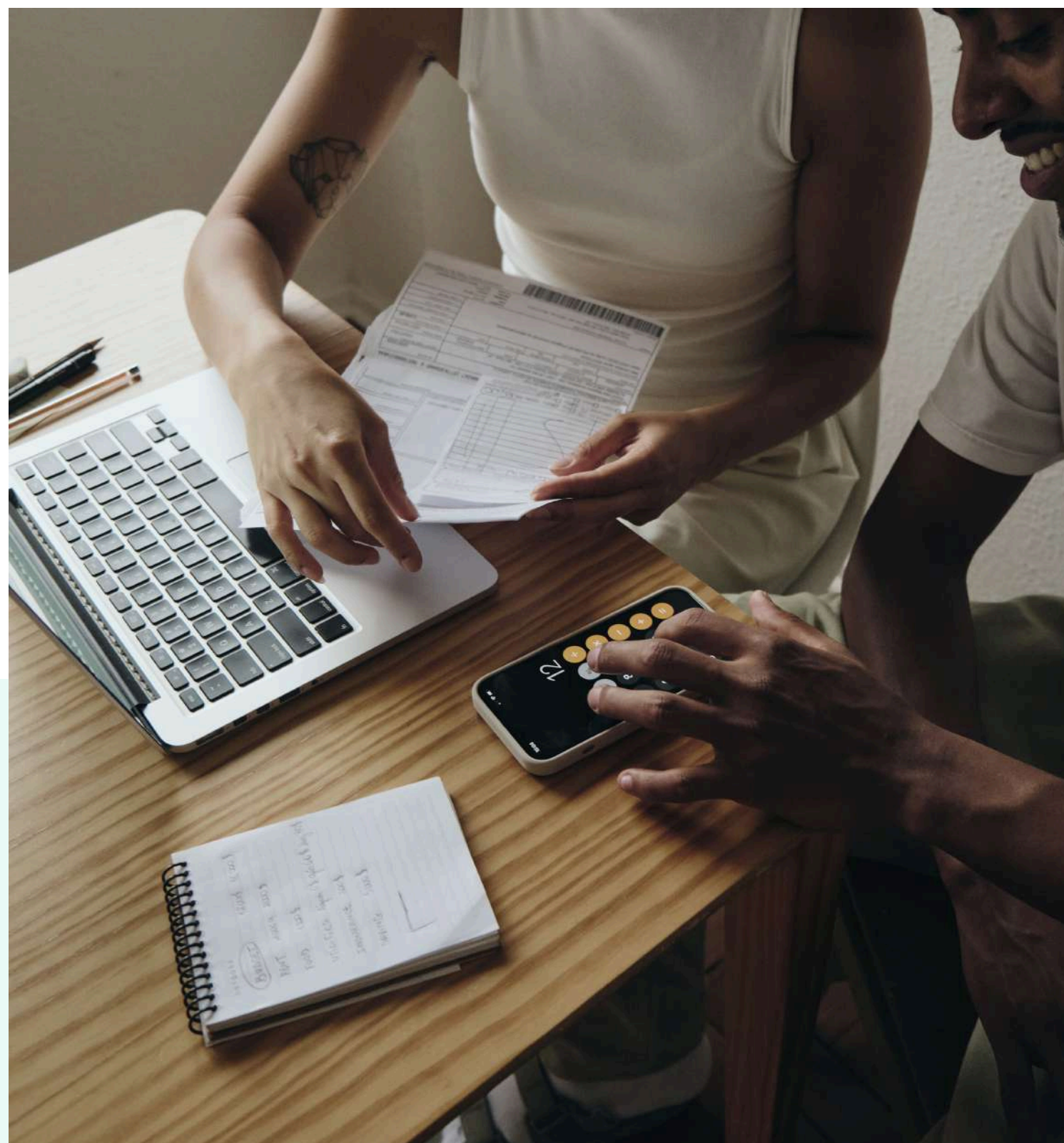
API Flask et déploiement



Déploiement automatique GitHub → Heroku

- API déployée sur Heroku
- Endpoint : https://credit-scoring-jademayalb-db8bcc609fed.herokuapp.com
- Route : /predict/<client_id>
- Test en temps réel d'un client





Présentation de l'analyse de data drift

Résultats

- Train vs Test : 32 variables importantes (SHAP)
- Drift global : Non détecté (< seuil 0.5)
- Variables avec drift : 3/32 (9.4%)

Dataset Drift

Dataset Drift is NOT detected. Dataset drift detection threshold is 0.5

32

Columns

3

Drifted Columns

0.0938

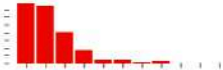
Share of Drifted Columns

Data Drift Summary

Drift is detected for 9.375% of columns (3 out of 32).

â€œ Search

×

Column	Type	Reference Distribution	Current Distribution	Data Drift	Stat Test	Drift Score
> AMT_GOODS_PRICE	num			Detected	Wasserstein distance (normed)	0.210785
> AMT_CREDIT	num			Detected	Wasserstein distance (normed)	0.207334
> AMT_ANNUITY	num			Detected	Wasserstein distance (normed)	0.161102

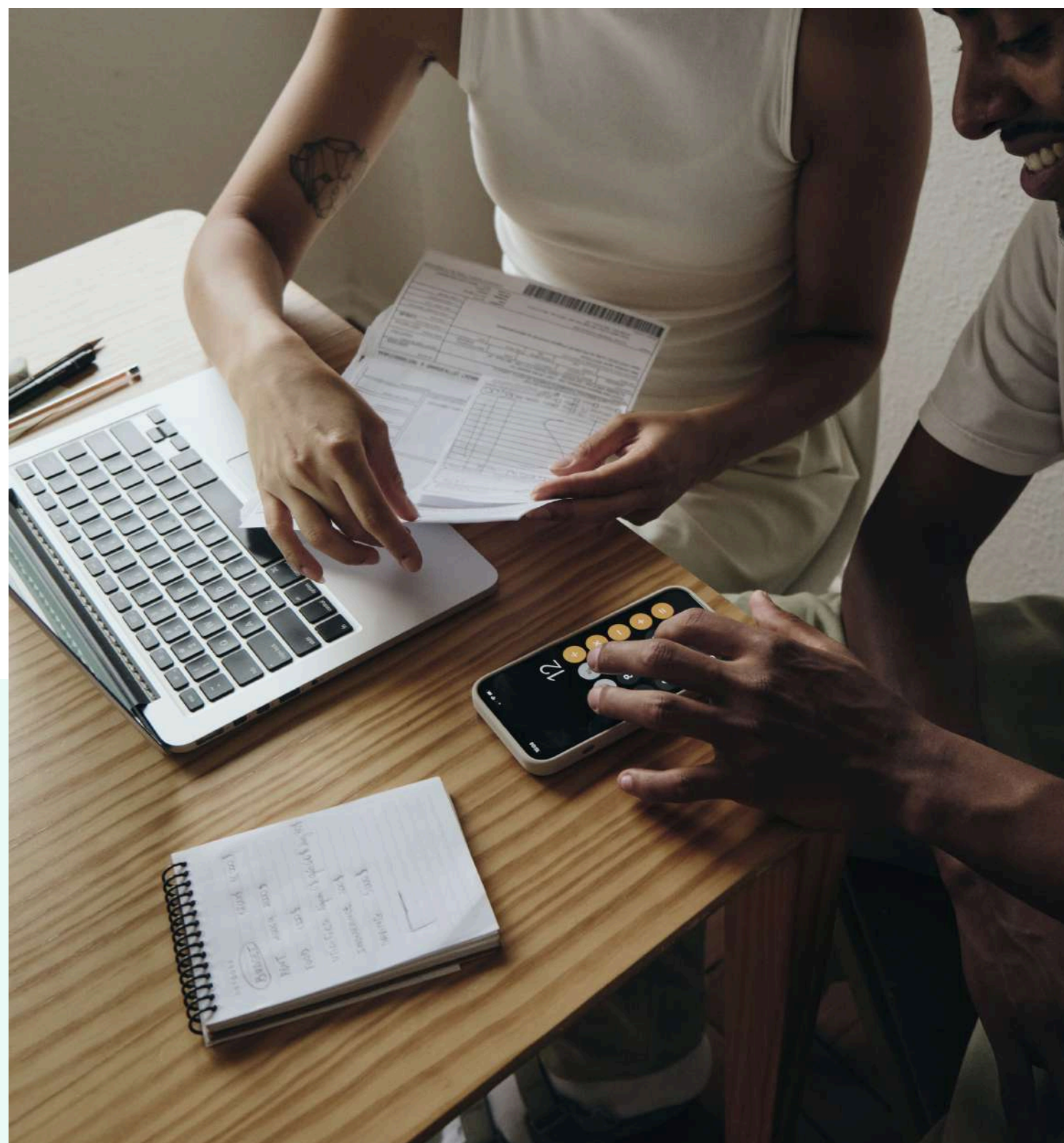
Variables impactées

- AMT_CREDIT, AMT_ANNUITY, AMT_GOODS_PRICE
- Variables critiques STABLES : EXT_SOURCE_1/2/3, DAYS_BIRTH

Stratégie monitoring

- Rapport HTML Evidently automatisé
- Surveillance renforcée variables financières
- Pipeline MLOps : alerte ré-entraînement si drift > seuil

Variable	Importance SHAP	Drift détecté
EXT_SOURCE_1	Haute	✗
EXT_SOURCE_2	Haute	✗
EXT_SOURCE_3	Haute	✗
DAYS_BIRTH	Haute	✗
AMT_CREDIT	Haute	⚠
AMT_GOODS_PRICE	Haute	⚠



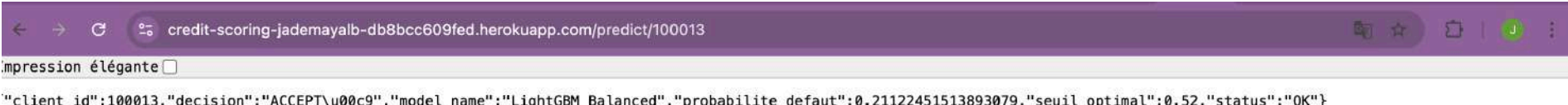
**Exemple d'un scoring
client via appel à l'API**

API sur Heroku

réponse JSON



<https://credit-scoring-jademayalb-db8bcc609fed.herokuapp.com/predict/100091>



<https://credit-scoring-jademayalb-db8bcc609fed.herokuapp.com/predict/100013>

Interface Streamlit

Scoring Crédit – Test de l'API

Choisissez l'identifiant client (SK_ID_CURR)

100091

Obtenir la prédiction

Décision : REFUSÉ

Probabilité de défaut : 0.523

Seuil optimal : 0.520

Deploy

Scoring Crédit – Test de l'API

Choisissez l'identifiant client (SK_ID_CURR)

100013

Obtenir la prédiction

Décision : ACCEPTÉ

Probabilité de défaut : 0.211

Seuil optimal : 0.520

Deploy



Conclusion

Limites

MLOps & monitoring

Drift détecté manuellement, versioning basique

API

Flask non sécurisé, scalabilité limitée

UX

Décisions "boîte noire" sans explication

Axes d'améliorations

Explicabilité & transparence

- Intégrer SHAP local dans l'API response
- Dashboard interactif pour conseillers
- Justification automatique des décisions

Monitoring automatisé

- Dashboard temps réel : drift + performance
- Alertes automatiques ré-entraînement
- MLflow Model Registry avancé

Production Ready

- API FastAPI + authentication + rate limiting
- Infrastructure cloud scalable (K8s)
- Interface utilisateur intuitive client/conseiller



Merci !